



Build Your Own Rocket Money

A Dustin OS white paper — how a small-business owner replaced a subscription-tracking app with a read-only bank feed, a local database, and a nightly check that lands in his morning email. No code required to follow.

Published by The Appraiser Coach, October 2026. Educational reference, written generically so it applies to any small business, not just an appraisal office. Also used as a case study in the Idaho Innovation Center AI workshops.

Read this first

- **This is a pattern, not a product.** There is no app to download. It describes something one person built for himself with an AI assistant, in enough detail that you could have yours build the same.
- **It is not financial, tax, or legal advice.** Connecting software to your bank accounts is your decision and your responsibility. Read your bank's terms and the terms of any service you use.
- **No warranty, no tech support.** If you build one, you own it.

Contents

1. Why replace an app that works
2. The whole thing in three boxes
3. Box one: the bank feed, and why not Plaid
4. Box two: the store on your own machine
5. Box three: the nightly check — what counts as a subscription
6. Where it lands: nowhere new
7. How it got built, including the part that went wrong
8. What it costs
9. If you want to build yours
10. Guardrails you should not skip

1. Why replace an app that works

Apps like Rocket Money, Monarch, and Copilot do a real job: they connect to your accounts, find the charges that repeat, total them up, and tell you when something looks off. If you run one business and have four accounts, use one and stop reading.

The case for building your own shows up when you have more than that. The person this paper is about runs five businesses out of twenty-three accounts at six banks. A consumer app sees all of that as one household. It cannot tell a payroll draw from a software subscription, it files the farm's electric bill next to a streaming service, and its "insights" are tuned to someone trying to cancel a gym membership, not someone trying to notice that a vendor quietly raised a weekly fee by twenty-six percent.

There are three quieter reasons too. The app is itself another subscription, which is a little funny for a subscription tracker. The app holds the connection to your banks, so if it is breached or sold, that is where the exposure is. And the app is a place you have to go. A dashboard you have to remember to open is a dashboard you will stop opening.

So the goal was narrow: know what repeats, catch double charges and price creep within a day or two, and see it in the one place that gets read every morning anyway. Nothing more.

2. The whole thing in three boxes

Every version of this, consumer app or homemade, is the same three boxes:

Box	What it does	In this build
Bank feed	Lets a program read your transactions without having your bank passwords.	SimpleFIN Bridge, a \$15-a-year service.
Store	Keeps every transaction somewhere you control, so you can ask questions about history.	One SQLite database file on a computer the owner already runs.
Nightly check	Looks at what is new, decides if anything is worth saying, and says it where you already look.	A small Python program on a timer, writing into the owner's existing morning email.

The important design rule, decided before anything was built: **nothing in the system can write to a bank**. Not a transfer, not a payment, not a cancellation. The bank feed chosen literally has no commands for that. A bug can at worst show you a wrong number. It cannot move a dollar.

3. Box one: the bank feed, and why not Plaid

When people say "connect to my bank," they usually mean Plaid. Plaid is excellent and it is what most consumer apps use. It is also built for companies: you sign up as a developer, request a plan, and pay per connected bank per month at a price they do not publish. For one person reading their own accounts it is a lot of paperwork and an unknown bill.

SimpleFIN Bridge does the same job for individuals. You subscribe, you log in to each of your banks on their site (they use a bank aggregator underneath, the same kind Plaid does), and when you are done you click one button that gives you a long one-time code called a *setup token*. Your own program trades that token, once, for a permanent read-only key. From then on the program asks the Bridge for transactions and the Bridge asks the banks. Your bank passwords never touch your machine. The Bridge has no write commands in its protocol at all, which is a stronger guarantee than a promise.

Practical notes from doing it:

- Six institutions linked in about fifteen minutes. The two-factor prompts from the banks are the slow part.
- Not every account is reachable. A card that lives inside a fintech app (in this case a crypto exchange's card) could not be linked. It was skipped, not fought. One card is not worth stalling the rest.
- The setup token works exactly once. If you paste it somewhere and it gets used, it is dead, and that is a feature. If the claim ever comes back "forbidden," revoke the token in the Bridge and make a new one rather than retrying.
- Transactions refresh about once a day on the Bridge's side, and some banks take a couple of days to post. This is a morning-after tool, not a real-time one, and that is fine for what it is for.

4. Box two: the store on your own machine

Every pull from the feed lands in a single database file. There is one table of accounts and one table of transactions, keyed by the bank's own transaction id, so pulling the same window twice just updates rows instead of duplicating them. That one decision is what makes the whole thing safe to rerun: if a night's job crashes halfway, the next night simply pulls the window again.

The first run backfilled a little over a year of history in 44-day slices, which is the Bridge's preferred request size. Nightly runs re-pull only the last 45 days, enough to catch a pending charge turning into a posted one. The file is readable only by the owner's account on that machine, and it is never copied anywhere.

The landmine worth writing down. Different feeds disagree on the sign of a number. In SimpleFIN a negative amount is money leaving. In Plaid a positive amount is money leaving. Pick one convention for

your store, write it at the top of the file, and convert on the way in. Get this backwards and every report runs perfectly while showing every total inverted.

5. Box three: the nightly check — what counts as a subscription

This is the part people assume needs machine learning. It does not. A recurring charge is just a pattern you can describe in a sentence, and the sentence is the code.

A subscription is: the same payee, charged at a regular interval, for a steady amount, at least three times. "Same payee" means the bank description with the junk stripped out (store numbers, the word POS, hashes). "Regular interval" means the gaps between charges cluster around a week, two weeks, a month, a quarter, or a year. "Steady amount" means the charges do not vary by more than about a third, which is what separates a software bill from your coffee shop. Transfers, deposits, card payments, and anything that looks like moving your own money around are excluded before any of that runs.

From there the inventory splits into two piles: **subscriptions** (software and services, the stuff you can cancel in five minutes) and **bills and loans on autopay** (payroll, insurance, lenders, utilities, anything over a certain size). Lumping those together is the mistake consumer apps make. A total of "\$20,000 a month in recurring charges" is useless when \$18,000 of it is payroll and the mortgage. "\$2,100 a month in software, here are the top five" is a number you can act on.

Then four checks, each a few lines:

1. **Double charge:** the same payee, the same amount to the cent, on the same account, twice within three days. Bank fees are excluded because they legitimately repeat.
2. **New recurring charge:** a pattern that qualifies as a subscription whose first charge was inside the last sixty days. This is how you find the free trial that started billing.
3. **Price jump:** a subscription whose latest charge is more than twenty percent above its usual amount.
4. **Broken feed:** the Bridge reported an error for a bank, which usually means that bank wants you to log in again.

Each finding gets a key, and the key is remembered. You are told once. If you do nothing, it does not nag you the next morning; whatever you do about it happens in your own task list, not in the money tool.

On the first run against real data it surfaced, in one pass: one probable double charge from a pest-control vendor, two price increases nobody had noticed, and a cloud-computing bill that had quietly started charging weekly a month earlier. That is the entire pitch for the thing in one sentence.

6. Where it lands: nowhere new

There is no dashboard. The nightly job writes a short text file only on nights it has something to say. The owner's existing morning email, which is generated by a separate program and already carries his calendar, weather, and to-do items, looks for that file. If it exists, the email gets a small "Money" section with those lines, worded as things to check. If it does not exist, the email says nothing about money and the reader never notices the silence.

Mondays add one line: how many subscriptions, roughly what they total, the five biggest, and the bills total separately. That is the whole reporting layer. It is deliberately boring.

If your morning place is a Slack channel, a notes app, or a text message to yourself, use that instead. The rule is to land where you already look, not to create a new place to look.

7. How it got built, including the part that went wrong

The build itself took one afternoon with an AI assistant that can write and run code on the owner's server. The owner's part was: subscribe to the Bridge, link the banks, paste the token, and then read the first inventory and say what looked wrong. The assistant's part was everything else: the sync, the store, the rules in section 5, the timer, the hook into the morning email, and testing each piece against the live feed as it went.

The honest part: this was the *second* attempt. The first time, three weeks earlier, the same system was designed and even coded in a chat session that did not have access to the server. It passed its tests inside that chat and then went nowhere, because the owner's one step ("subscribe and paste a token") was written into a journal as a note instead of into his to-do list as a task. He asked his assistant what to work on, was told "nothing pressing," and built a video game instead. Three weeks later he asked where the money tool was.

Two lessons came out of that, and they are worth more than the tool:

- **A handoff is not done until it is written as the other person's task, in their list, in a verb.** "Still needs Dustin to subscribe" in a note is invisible. "Subscribe at this link and paste the token" in his to-do list gets done.
- **Build on the machine where it will run.** Code that passes tests somewhere else is a design, not a tool.

The only real bug on the second attempt: the Bridge refused the token claim with a "forbidden" error when the request came from Python's default web client, and accepted it from a plain `curl` command. The fix was sending a normal user-agent header. It cost ten minutes and is noted here so it costs you zero.

8. What it costs

- **SimpleFIN Bridge:** \$15 a year.
- **A computer that is on at night:** the owner already had a small home server. A Raspberry Pi, an old laptop, or a \$5-a-month cloud box all work.
- **The AI assistant:** whatever you already pay for one that can run code (the owner uses Claude Code on a monthly plan).
- **Your time:** about forty-five minutes of yours across an afternoon, most of it bank logins.

9. If you want to build yours

- 1 **Decide where it will land.** The email you read every morning? A Slack channel? If you do not have a daily place, start with a plain text file and read it with your coffee.
- 2 **Subscribe to SimpleFIN Bridge** at bridge.simplefin.org and link your accounts. Skip anything that will not connect. Do not generate the setup token yet.
- 3 **Describe the tool to your AI assistant** in your own words. Something like the box below. Ask for it to be built on the machine that will run it, in a scratch folder, and to show you the first inventory before anything is scheduled.
- 4 **Generate the setup token and give it to the assistant** through the tool it is building, never by pasting it into a public place. Confirm it stored the resulting key in a file only your user can read.
- 5 **Read the first inventory.** It will be wrong in a few places: a loan filed as a subscription, a coffee shop that happens to bill like clockwork. Say so. Each correction is one line of rule.
- 6 **Turn on the timer only when the output is boring.** If the first week's mornings have a false alarm, fix the rule, not your tolerance.

I want a private replacement for a subscription-tracking app.
Read-only. It must never be able to move money.

Bank feed: SimpleFIN Bridge (I'll give you a one-time setup token).

Store: a local SQLite file on this machine, readable only by me.

Nightly: pull the last 45 days, update the store, then find

- recurring charges (same payee, regular interval, steady amount, 3+ times),
split into "subscriptions" and "bills/loans on autopay",
- the same charge twice within 3 days,
- any subscription whose first charge is within 60 days,
- any subscription that jumped more than 20%,

- any bank the feed reports an error for.

Tell me once per finding. Write findings to a file my morning
[email / Slack / note] already reads; write nothing on quiet nights.
Mondays: one line with totals and the five biggest subscriptions.
Build it here, in a scratch folder, test against the live feed,
and show me the first inventory before scheduling anything.

10. Guardrails you should not skip

- **Read-only or nothing.** If a feed or a tool can initiate transfers, do not use it for this, no matter how convenient.
- **The access key is a password.** Keep it in one file, permissions locked to your user, never in a code repository, never in a chat log, never in a screenshot.
- **Keep the store on a machine you control** and do not sync that folder to a cloud drive by accident.
- **Treat bank-supplied text as untrusted.** Transaction descriptions and error messages come from outside. Cap their length and never feed them to anything that executes.
- **Tell the humans.** If accounts belong to a spouse or a business partner, they should know a program on your machine reads them.
- **Silence is not success.** Make the nightly job say something when the feed itself fails. A money tool that quietly stops is worse than no tool.

Repeat of the disclaimer

- Pattern, not product. No software is distributed with this paper.
- Not financial, tax, or legal advice. Connecting software to financial accounts is your responsibility, including compliance with your banks' terms and any third-party service's terms.
- Figures in this paper are rounded and illustrative. No account details are shared.